Proceedings of the 2001 IEEE
International Conference on Robotics & Automation
Seoul, Korea · May 21-26, 2001

Spanning-Tree Based Coverage of Continuous Areas by a Mobile Robot

Yoav Gabriely and Elon Rimon
Dept. of Mechanical Engineering
Technion, Israel Institute of Technology
meeryg@tx.technion.ac.l,elon@robby.technion.ac.il

Abstract *This paper considers the problem of covering a continuous planar area by a square-shaped tool attached to a mobile robot. Using a tool-based approximation of the work-area, we present an algorithm that covers every point of the approximate area. The algorithm, called* Spanning Tree Covering *(STC), subdivides the work-area into disjoint cells corresponding to the square-shaped tool, then follows a spanning tree of the graph induced by the cells, while covering every point precisely once. We present and analyze three versions of the STC algorithm. The first version is an off-line algorithm that computes an optimal covering path in linear time $O(N)$, where N is the number of cells comprising the approximate area. The second version is an on-line or sensor based algorithm, that completes an optimal covering path in time $O(N)$, but requires $O(N)$ memory for its implementation. The third version of STC is "ant"-like, where the robot may leave pheromone-like markers during the coverage process. The ant-like STC algorithm runs in time $O(N)$ and requires only $O(1)$ memory. We present simulation results of the three STC algorithms, demonstrating their effectiveness in cases where the tool size is significantly smaller than the work-area characteristic dimension.*

1 Introduction

The *mobile robot covering problem* can be formulated as follows. Let a tool of a specific planar shape be attached to a mobile robot, and let $\mathcal{A}$ be a continuous planar work-area bounded by obstacles. Then the mobile robot has to move the tool along a path such that every accessible point of $\mathcal{A}$ is covered by the tool along the path. The covering problem is receiving considerable attention due to current interest in competitive algorithms[1] for autonomous systems that operate with incomplete information [11]. The optimal covering problem can be formulated as a generalization of the Traveling Salesperson Problem (TSP) for a continuous domain, and thus is NP-hard [1]. It is therefore natural to seek competitive algorithms for the covering problem.

The mobile robot covering problem has several important applications. One such application area is automatic floor cleaning and coating in facilities such as supermarkets and train stations [6]. Other application areas are lawn mowing [14], hazardous waste cleaning [8], and field demining [12]. Furthermore, although we focus on mobile robot coverage, the same methods apply to other robotic coverage tasks such as part machining [9]. Note that map building or area exploration is *distinct* from the covering problem considered here. In the map building problem an autonomous robot has to completely scan an unknown environment using its sensors, e.g. [10, 16]. Map building can be interpreted as achieving sensory coverage of an unknown environment. In contrast, area coverage requires physical sweeping of a tool over every point of a given work-area.

Previous efforts on the covering problem have focused mainly on the *discrete* version of the problem, where a robot has to visit all nodes of an unknown graph by traversing its edges [4, 5, 18]. The *continuous* covering problem, where a robot must sweep a tool over a continuous area, has been studied by Ntafos [13]. Much like in our approach, Ntafos imposes a tool-based grid approximation over the work-area. Then he subdivides the grid into rectangular sub-grids that can be covered using a TSP path that visits all sub-grids. Ntafos considers only simple grids with no internal holes, and his algorithm covers the grid within 33% of its optimal covering path. The continuous covering problem is also studied by Arkin et al. [1]. They propose an algorithm that covers non-simple grids which possess no local cut nodes[2] within 32.5% of the optimal covering path. In contrast, we consider grids with double-size cells, and our algorithm produces optimal covering paths for these grids. Furthermore, the algorithms of Ntafos and Arkin are strictly off-line while ours has an on-line version. The contin-

[1] An algorithm is *competitive* if its solution to every problem instance is a constant times the optimal solution to the problem with full information available.

[2] A *local cut node* is a node whose removal locally disconnects the graph induced by the grid.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 05,2026 at 18:57:14 UTC from IEEE Xplore. Restrictions apply.

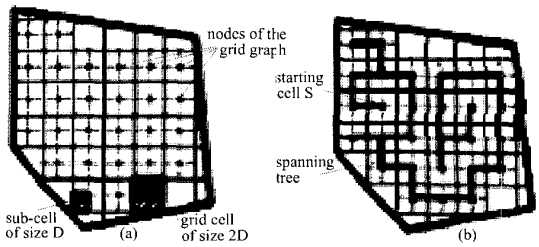

Figure 1: (a) Grid approximation of a given work-area. (b) A spanning tree for the grid.

uous coverage problem is also studied by Butler at al. [2], Choset and Pignon [3], and Pirzadeh and Snyder [15]. All of these works emphasize the importance of achieving on-line sensor based coverage.

We make the following three assumptions. First, we assume that the tool is a square of size D. Second, the robot is allowed to move the tool only in directions orthogonal to the tool's four sides, without rotating the tool. Third, we subdivide the work-area into square cells of size $2D$, and discard cells which are partially covered by obstacles (Figure 1). As later discussed with the analysis and simulations, the quality of approximation depends on the tool size D being significantly smaller than the work-area characteristic dimension. The algorithm, called *Spanning Tree Covering* (STC), follows a spanning tree of the graph induced by the cells, while covering every point precisely once. However, unlike previous covering paradigms, we carefully weave the traversal of the spanning tree with the covering of individual cells. This integration of inter-cell traversal with intra-cell coverage is a novel approach that has not appeared in the literature before.

We present and analyze three STC algorithms. The first is an off-line algorithm, that computes an optimal covering path in linear time $O(N)$, where N is the number of cells. The second is an on-line STC algorithm, that has no apriori knowledge of the environment and completes an optimal covering path in time $O(N)$. However, it requires $O(N)$ memory, and this requirement limits the size of work-areas that can be covered by the on-line algorithm. The third STC algorithm is "ant"-like. In this version, too, the robot has no apriori knowledge of the environment, but it may leave markers during the coverage process. The ant-like STC algorithm completes an optimal covering path in time $O(N)$, and requires only $O(1)$ memory. The analysis section also contains an adaptation of the on-line STC to an algorithm that achieves *exact* coverage of the work-area. The exact STC algorithm is amenable to competitive analysis, and we show that it achieves complete coverage with a competitive ratio of $1+3(\Pi D/A)$, where D is the tool size, Π is a quantity related to the obstacles' total perimeter, and A the total area. In practice $\Pi D << A$, and STC typically achieves an almost optimal coverage of the work-area. Finally we present simulation results of the STC algorithms, demonstrating their effectiveness in cases where the tool size is significantly smaller than the work-area characteristic dimension.

2 The STC Algorithm

In this section we describe three versions of the *Spanning Tree Covering* (STC) algorithm.

2.1 The Off-Line STC Algorithm

The input to the off-line STC algorithm is a geometrical description of a planar environment bounded by piecewise-smooth and stationary obstacles. The algorithm first subdivides the work-area into cells of size $2D$, and discards cells which are partially covered by obstacles. We define a graph structure, G, with nodes at the center points of the cells, and edges that connect centers of adjacent cells (Figure 1(a)). The algorithm next constructs a spanning tree for G (Figure 1(b)), and uses this tree to generate a covering path as follows.

Off-line STC Algorithm:

Input: A geometrical description of the environment, converted to a $2D$-size grid with a graph structure G as described above. A starting cell S.

Pre-processing: Starting from S, construct a *spanning tree* for G using any spanning-tree construction algorithm. Subdivide every $2D$-size cell into four identical *subcells* of size D.

Covering action: Starting at a subcell of S, move between neighboring subcells (each being identical to the tool's shape) along a path which *circumnavigates the spanning-tree along a counterclockwise direction.* Halt when the starting subcell is encountered again.

An execution example is illustrated in Figure 2. It can be seen that the robot moves the covering tool through subcells that lie on the right-side of the spanning-tree edges, measured with respect to the tool's direction of motion. The figure possesses two unrealistic features, which were added for clarity. The tool size is shown unrealistically large with respect to the work-area size, and the covering path is shown curved while it is actually rectilinear.

2.2 The On-Line STC Algorithm

In this version of STC the robot has no prior knowledge about the environment. Rather, the robot has position and orientation sensors that allow it to lo-

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 05,2026 at 18:57:14 UTC from IEEE Xplore. Restrictions apply.

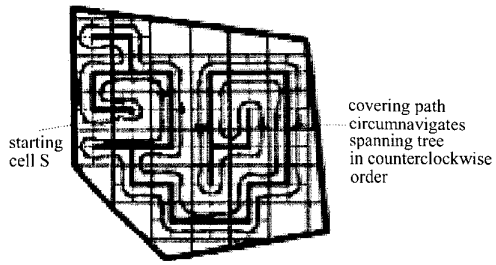

Figure 2: An execution example of the off-line STC.

cally recognize the $2D$-size cells comprising the work-area. We also assume a range sensor, capable of identifying obstacles in the four cells neighboring the robot's current cell (Figure 3(a)). The important issues of sensor measurement errors and sensor fusion are not considered here. Rather, we assume ideal sensors that provide perfect readings. In the on-line STC algorithm, the robot incrementally constructs the work-area grid by subdividing every cell it encounters into four identical subcells of size D. At the same time, the covering tool follows a subcell path which circumnavigates the incrementally constructed spanning tree, until the entire work-area grid is covered. A description of the algorithm follows.

On-line STC Algorithm:

Sensors: A position and orientation sensor. An obstacle detection sensor.

Input: A starting cell S, but no apriori knowledge of the environment.

Recursive function: $STC(w, x)$, where x is the current cell and w the parent cell in the spanning tree.

Initialization: Call $STC(Null, S)$, where S is the starting cell.

$STC(w, x)$:

1. Mark the current cell x as an *old* cell.
2. While x has a *new* obstacle-free neighbor:
2.1 Scan for first new neighbor of x in counterclockwise order, starting with parent cell w. Call this neighbor y.
2.2 Construct a spanning-tree edge from x to y.
2.3 Move to a subcell of y as described below.
2.4 Execute $STC(x, y)$.

End of while loop.

3. If $x \neq S$, move back from x to a subcell of w as described below.
4. Return. (End of $STC(w, x)$.)

We now discuss several details of the algorithm. First, the robot runs a DFS algorithm during the incremental spanning-tree construction. The counterclockwise scanning of neighbors specified in step 2 is

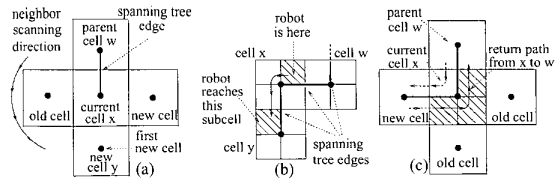

Figure 3: (a) Counterclockwise scanning of four neighbors. (b) A move from x to a new cell y. (c) A return from x to a parent cell w.

crucial for ensuring that the covering tool circumnavigates the incrementally constructed spanning tree in counterclockwise order (Figure 3(a)). Second, in step 2.3 the covering tool is located in a subcell of x and has to move into a new cell y. By construction there is already a spanning-tree edge from x to y. The covering tool moves from its current subcell in x to a subcell of y by following the right-side of the spanning tree edges, measured with respect to the tool's direction of motion (Figure 3(b)). It is shown below that the robot covers the subcells of x in counterclockwise order. Since the robot selects the first new neighbor y in counterclockwise order, the covering tool always has a subcell path from x to y which follows the right-side of the spanning tree edges. When the covering tool returns to a parent cell w, it again moves through subcells that lie on the right-side of the spanning-tree edge connecting x with w (Figure 3(c)).

2.3 The Ant-Line STC Algorithm

The ant-like STC is also an on-line algorithm, that uses the same sensors described above. However, now the robot can leave markers in the subcells it covers, using color, odor, heat, or pebble markers [17]. We additionally assume the availability of a detection device, capable of inspecting the subcell markers in the current cell and its four neighbors. Using this detection device, the robot identifies a neighboring cell as *new* when its four subcells are all unmarked.

Much like the on-line STC algorithm, here too the robot uses DFS to incrementally construct a spanning tree for the work-area grid. However, rather than store the spanning tree in its memory, the robot uses the subcell markers to identify parent cells along the spanning tree as follows. By construction, the robot covers and marks the subcells of a given cell in counterclockwise order. Hence as long as the four subcells of the current cell are not all marked, the robot need only scan the subcells in counterclockwise order, and identify the unique transition between unmarked and marked subcells along a counterclockwise direction (Figure 4). The two subcells bordering this transition have an exterior edge in common, and this

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 05,2026 at 18:57:14 UTC from IEEE Xplore. Restrictions apply.

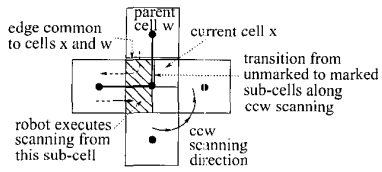

Figure 4: Parent detection using subcell markers.

edge is necessarily the boundary edge between the current cell and its parent cell in the spanning tree. A description of the algorithm follows.

Ant-like STC Algorithm:
Marking device: A device that marks the subcell currently being covered.
Sensors: A position and orientation sensor. An obstacle detection sensor. A marker detector.
Input: A starting cell S, but no apriori knowledge of the environment.
Find parent function: $Parent(x)$ which returns the parent of the current cell x in the spanning tree, using the technique described above.
Find new neighbor function: $Neighb(w, x)$ which accepts as inputs the current cell x and its parent cell w, and returns the first *new* obstacle-free neighbor of x in counterclockwise order, starting with w. Returns *Null* if x has no new neighbor.
Initialization: Set $x = S$.
While S has unmarked subcells:
1. Determine the parent cell of x, $w = Parent(x)$.
2. Determine a *new* neighbor of x, $y = Neighb(w, x)$.
3. If y is not *Null*:
Move to a subcell of y, while marking the subcells of x being covered. Set x=y.
4. Else (y is *Null*):
Return from x to a subcell of w, while marking the subcells of x being covered. Set $x = w$.
End of while loop.

We now discuss several details of the algorithm. First, in the ant-like STC the robot does not explicitly construct the spanning tree. Rather, it computes the two spanning-tree edges required for the local motion of the covering tool. The edge from the current cell x to its parent cell is computed with $Parent(x)$. The edge from x to a new neighbor is computed from knowledge of the neighbor. Next consider the subcell marking specified in the algorithm. $Parent(x)$ can identify the parent cell of x only when not all four subcells of x are marked. Hence *the robot marks a subcell only when the covering tool leaves this subcell.* This feature allows the robot to identify the parent cell of w in the next step of the algorithm.

3 Algorithm Analysis

In this section we analyze the three STC algorithms, then present a version of the on-line STC algorithm which is amenable to competitive analysis.

3.1 Properties of the STC Algorithm

For purposes of analysis, we define a *repetitive coverage* of a point p as the situation where p is being covered by the robot's tool, then exposed, and later covered again. We first establish that the three STC algorithms do not generate any repetitive coverage.

Lemma 3.1. *The three STC algorithms do not repetitively cover any work-area point.*

Proof sketch: The robot guides the tool between adjacent subcells using translational motion which is perpendicular to the subcells' edges. Since each subcell is identical to the tool in shape and size, motion between adjacent subcells involves no repetitive coverage of either subcell. Since the covering tool visits every subcell precisely once during the spanning-tree circumnavigation (Figure 2), every point is covered precisely once. □

The following proposition considers the covering pattern of individual cells.

Proposition 3.2. *The three STC algorithms cover the subcells of any particular cell in a counterclockwise (but not necessarily contiguous) order.*

Proof sketch: The covering tool moves along a subcell path that locally follows the right-hand side of the spanning tree, measured with respect to the tool's direction of motion. If a cell x has no new neighbors, x is a leaf-node of the spanning tree, and the tool circumnavigates this node in counterclockwise order as depicted in Figure 5(a). If x has new neighbors, the robot selects the *first* neighbor in counterclockwise order, starting with the parent cell of x. Let y denote the first new neighbor. The covering tool next follows the right-side of the spanning tree to a subcell of x which allows it to exit into y. As depicted in Figure 5(b)-(d), motion along the right-side of the spanning tree implies that the tool moves through subcells of x in counterclockwise order. □

The proposition ensures that the covering tool can always access a new neighbor of the current cell x through empty subcells of x (Figure 5). The proposition also ensures that all cells are covered by the *same* counterclockwise pattern that starts with a subcell adjacent to the parent cell. The ant-like STC uses this coverage pattern to uniquely identify the parent cell using subcell markers. The next lemma asserts the completeness of the STC algorithms [7].

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 05,2026 at 18:57:14 UTC from IEEE Xplore. Restrictions apply.

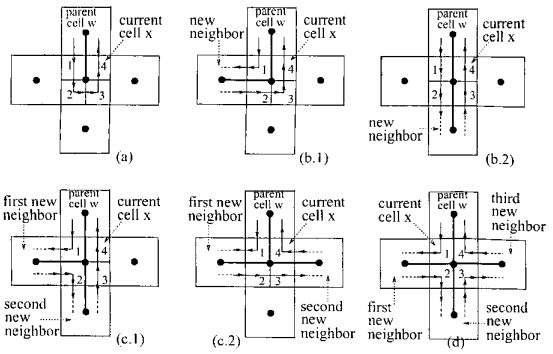

Figure 5: The subcell paths associated with (a) a cell with no new neighbors, (b)-(c) a cell with one or two new neighbors, (d) a cell with three new neighbors.

Lemma 3.3. *The three STC algorithms cover every cell which is accessible from the starting cell S.*

The following theorem summarizes the coverage properties of the STC algorithms.

Theorem 1. *The three STC algorithms completely cover the continuous area underlying the accessible work-area grid. Moreover, the covering is optimal in the sense that there is no repetitive coverage of any point in this area.*

The theorem follows from Lemma 3.1 and Lemma 3.3. The next theorem lists several performance bounds for the STC algorithms.

Theorem 2. *Let N be the number of cells in the accessible work-area grid. Then the three STC algorithms cover this area in $O(N)$ steps, using a covering path of length $L = 4ND$ where D is the tool size. Moreover, the on-line STC requires $O(N)$ memory, while the ant-like STC requires $O(1)$ memory.*

Let us sketch the derivation of these bounds. The covering path is a rectilinear curve, and in each covering step the tool traverses a distance D between adjacent subcells. Since there are $4N$ subcells, $L = 4ND$. The $O(N)$ memory requirement of the on-line STC algorithm allows it to identify which cells of the grid have already been visited by the covering tool. In contrast, the ant-like STC stores only two $O(1)$ memory items. The first is the coverage state of the starting cell S, which allows the algorithm to determine completion of coverage. The second is the identity of the spanning-tree edge leading to the parent cell. This knowledge allows the robot to uniquely identify the four subcells of the current cell, which in turn allows the robot to properly scan the neighbor cells.

3.2 Competitive Analysis

We now adapt the on-line STC algorithm to an algorithm called *exact STC* that achieves exact coverage. Then we compare the covering path generated by the exact STC algorithm to the optimal covering path with full information available. First we make two assumptions. We assume that the tool size D is sufficiently small so that the connectivity of the work-area is preserved by the grid approximation. This assumption is satisfied when the minimal gap between obstacles in the environment is at least $2R$, where $R = 2\sqrt{2}D$ [7]. Moreover, any point of the work-area not included in the grid approximation lies at a distance of at most R from one of the obstacles. Hence, in order to achieve exact work-area coverage the robot must augment the grid coverage with a sweep of a thickness-R neighborhood about the obstacles. We also assume that the covering tool can freely rotate while following the contour of an obstacle boundary. The exact STC algorithm simply runs the on-line STC algorithm with the following modification. Each time the current cell x has a neighbor cell occupied by an obstacle, the robot checks if this is a new obstacle. If it is a new obstacle, the robot halts the grid coverage and sweeps a thickness-R neighborhood about the obstacle. During this sweep, the robot marks in its $O(N)$ memory the obstacle boundary as being already covered. The robot completes the sweep at the cell x, and from this cell it resumes the grid coverage according to the on-line STC.

The covering path generated by the exact STC algorithm consists of segments that correspond to the grid coverage, and segments associated with the sweep around obstacles. The total length of the segments of the first type is $4ND$ according to Theorem 2. The total length of the segments of the second type is as follows. Since $R = 2\sqrt{2}D$, the coverage of a thickness-R neighborhood about the obstacles requires *three* passes at distances of $D/2$, $3D/2$, and $5D/2$ from the obstacles' boundaries. Under fairly general conditions the length of the curve at a distance $3D/2$ is the average of the lengths of the other two curves [7]. Hence the total length of the segments of the second type is 3Π, where Π is the total length of the curves at a distance $3D/2$ from the obstacles. Our next task it to derive a lower bound on the length of the optimal covering path. In order to simplify the derivation, we assume that the covering tool moves along a piecewise linear path, maintaining a fixed orientation along each linear segment.

Lemma 3.4 (optimality bound [7]). *Let a continuous and connected work-area have a total area A. Then the length L of any piecewise-linear covering*

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 05,2026 at 18:57:14 UTC from IEEE Xplore. Restrictions apply.

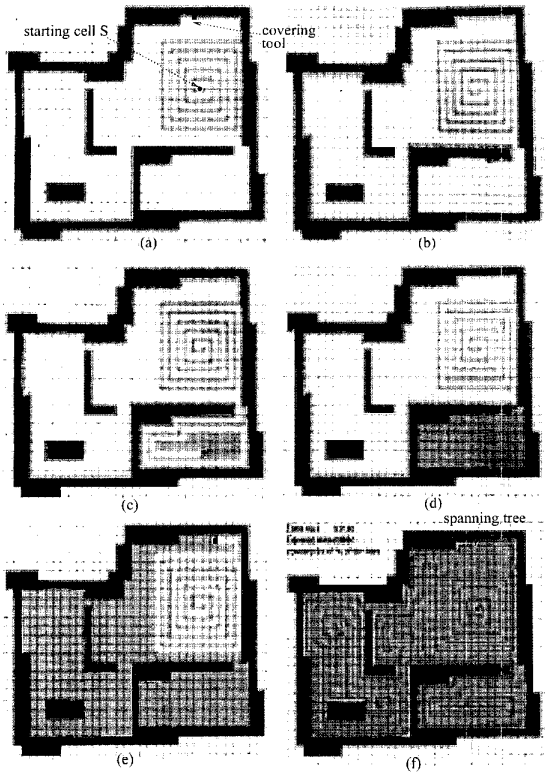

Figure 6: Six covering stages of the on-line and ant-like STC algorithms.

path of the work-area satisfies $L \geq A/D_{\max}$, where $D_{\max}$ is the tool's maximal sweeping width.

In our case the covering tool uses a fixed sweeping width of D. The total length of the covering path generated by the exact STC algorithm is $4ND + 3\Pi \leq A/D + 3\Pi$, since $4ND^2 \leq A$. It follows that the *competitive ratio* of the exact STC algorithm is: $(A/D + 3\Pi)/(A/D) = 1 + 3\Pi D/A$. In practice $\Pi D << A$, and the exact STC algorithm typically achieves an almost optimal coverage of the work-area. However, the exact STC algorithm may perform poorly in certain congested environments, due to its inefficient sweep of a thickness-R neighborhood about the obstacles. Ref. [7] describes such environments.

4 Simulation Results

In this section we present simulation results of the on-line and ant-like STC algorithms. The first example runs the two algorithms on the environment shown in Figure 6. The two algorithms always generate identical covering paths, and the covering stages shown in Figure 6 represent the execution of both algorithms. However, the on-line STC maintains a data structure of the incrementally constructed spanning tree, while the ant-like STC uses subcell markers to locally identify the spanning tree. In Figure 6(a) the covering tool follows the right-side of the spanning tree edges along a subcell path which spirals outward from the starting cell S. In Figure 6(b) the covering tool has reached the outer boundary, and it follows this boundary until it reaches the entrance to the lower-right room. In Figure 6(c) the covering tool spirals inward to the center of the lower-right room, and in Figure 6(d) it spirals outward along the complementary spiral. At this stage the lower-right room is completely covered. The covering tool next proceeds to the left portion of the environment. This portion is populated by two obstacles that divide the area into three cavities. Each of these cavities is covered by the same double-spiral pattern, and the resulting coverage is shown in Figure 6(e). Finally, the covering tool spirals inward to the starting cell S, resulting in a complete coverage of the work-area grid. The spanning tree traced by the on-line STC and ant-like STC algorithms appears in Figure 6(f). It should be noted that the covering path is optimal, in the sense that there is no repetitive coverage of any point in the area underlying the grid.

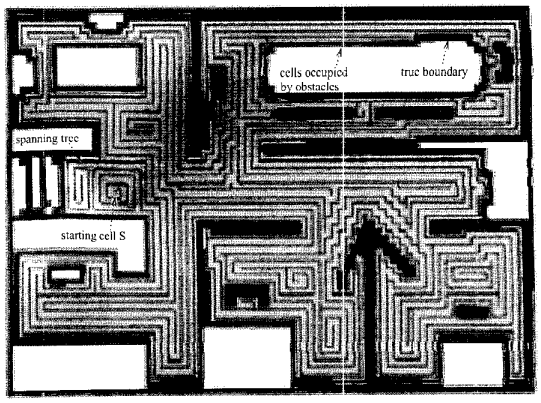

Figure 7: Office environment coverage by a tool size an order-of-magnitude smaller than a door's width.

A second example, shown in Figure 7, attempts to provide a realistic office environment of a startup company making robotic vacuum-cleaners. This example consists of several rooms and offices populated by pieces of furniture. The tool size D is selected to be an order-of-magnitude smaller than a door's width. The execution of the on-line STC and ant-like STC algorithms on 100 instances of this environment yields a 85%-95% coverage of the total area. In the

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 05,2026 at 18:57:14 UTC from IEEE Xplore. Restrictions apply.

particular office environment shown in Figure 7, the covered area is 90% of the total area. Note that the portions of the work-area not covered by the STC algorithms lie around the obstacle perimeters. Hence it is possible to augment the grid coverage with a sweep around the obstacles as described above, in order to achieve coverage of the entire work-area.

5 Conclusion

We presented and analyzed three spanning-tree based covering algorithms for continuous work-areas. The algorithms impose a $2D$-size grid approximation on the given work-area, where D is the tool size. All three algorithms use the same procedure of constructing a spanning tree for the grid, then following a subcell path that circumnavigates the spanning-tree edges. The resulting subcell path completely covers the work-area grid in optimal time $O(N)$, where N is the number of grid cells. The quality of coverage depends on the tool size being significantly smaller than the gaps between obstacles in the environment. Our simulations show that in typical office environments a tool size which is an order-of-magnitude smaller than a door's width yields an average cover of 90% of the work-area.

Each of the three STC algorithms has merits and caveats. The off-line STC algorithm requires apriori information about the environment. However, this algorithm is able to pre-compute spanning trees that guide the covering tool along desired patterns, such as parallel-line scanning. The on-line STC algorithm covers an environment whose geometry is not apriori known to the robot, and is therefore suitable for office and home settings. However, it requires $O(N)$ memory, which imposes a limit on the size of the work-areas that can be covered by a limited-memory robot. The ant-like STC algorithm uses subcell markers to achieve complete coverage with only $O(1)$ memory. However, subcell marking mechanisms are still in a development stage and are not readily available. Finally, we also described a fourth STC algorithm that augments the grid coverage with a sweep around the obstacles' boundaries. The latter algorithm achieves an exact coverage with a competitive ratio of $1 + 3(\Pi D/A)$, where Π is related to the obstacles' perimeter and A is the total area. In typical office-like environments $\Pi D << A$, and in these environments the augmented STC algorithm achieves an almost optimal coverage of the entire work-area.

References

[1] E. M. Arkin, S. P. Fekete, and J. S. Mitchell. Approximation algorithms for lawn mowing and milling. Tech. report 97.255, University of Koeln, 1997.

[2] Z. J. Butler, A. A. Rizzi, and R. L. Hollis. Complete distributed coverage of rectlinear environments. *4'th WAFR*, Hanover, New Hampshire, 2000.

[3] H. Choset and P. Pignon. Coverage path planning: boustrophedon decomposition. *Int. Conf. on Field and Service Robotics*, Canberra, Australia, 1997.

[4] X. Deng and A. Mirzaian. Competitive robot mapping with homogeneous markers. *IEEE Trans. on Robotics and Automation*, 12(4):532–542, 1996.

[5] G. Dudek, M. Jenkin, E. Milios, D. Wilkes. Robotic exploration as graph construction. *IEEE Trans. on Robotics and Automation*, 7(6):859–865, 1991.

[6] H. Enders, W. Feiten, and G. Lawitzky. Field test of Autonomous cleaning in supermarkets. *IEEE Int. Conf. on Robotics and Automation*, 1779–1781, 1998.

[7] Y. Gabriely and E. Rimon. Spanning-tree coverage by a mobile robot. Tech. report, Technion, http://www.technion.ac.il/~robots, Dec. 1999.

[8] S. Hedberg. Robots cleaning up hazardous waste. *AI Expert*, pages 20–24, May 1995.

[9] M. Held. *On the Computational Geometry of Pocket Machining*. Springer-Verlag, New York, 1967.

[10] S. Hert, S. Tiwari, and V. J. Lumelsky. A terrain covering algorithm for an AUV. *Autonomous Robots*, 3:91–119, 1996.

[11] C. Icking and R. Klein. Competitive strategies for autonomous systems. In *Modelling and Planning for Sensor Based Intelligent Robot Systems*, pages 23–40. World Scientific, Singapore, 1995.

[12] J. D. Nicoud and M. K. Habib. The PEMEX autonomous demining robot. *IEEE/RSJ Int. Conf. on Intelligent Robot Systems*, pages 1:419–424, 1995.

[13] S. Ntafos. Watchman routes under limited visibility. *Computational Geometry: Theory and Applications*, 1:149–170, 1992.

[14] U. Peles and S. Abramson. *RoboMow by Friendly Machines*. www.friendlymachines.com, Israel, 1998.

[15] A. Pirzadeh and W. Snyder. A unified solution to coverage and search in explored and unexplored terrains using indirect control. *IEEE Int. Conf. on Robotics and Automation*, pages 2113–2119, 1990.

[16] N. S. V. Rao and S. S. Iyengar. Autonomous robot navigation in unknown terrains: visibility graph based methods. *IEEE transactions on Systems, Man and Cybernetics*, 20(6):1443–1449, 1990.

[17] R. A. Russell. Laying and sensing odor markings for assisting mobile robot navigation tasks. *IEEE Robotics and Automation Mag.*, 2:3–9, Sep. 1995.

[18] I. A. Wagner, M. Lindenbaum, and A. M. Bruckstein. Distributed covering by ant-robots using evaporating traces. *IEEE Transactions on Robotics and Automation*, 15(5):918–933, 1999.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 05,2026 at 18:57:14 UTC from IEEE Xplore. Restrictions apply.